

NATIONAL ECONOMICS UNIVERSITY

Faculty of Data Science and Artificial Intelligence

STATISTICS

Machine Learning-Based Electricity Consumption Forecasting in Tetouan City

---

A chronological evaluation of conventional regression models

GROUP MEMBERS

No.	Full Name	Student ID
1	Dang Nhat Minh	11247196
2	Giang Hoang Son	11247222

Class: DS66A
Instructor: Ph.D. Luong Van Thien

Hanoi, 2026

Abstract

Distributors need a forecast of every hour of the next day to plan electricity purchases. This project forecasts the hourly load of Tetouan City (sum of three distribution zones, 2017) one day ahead with five conventional models (linear regression, decision tree, random forest, SVR and XGBoost) and compares them with seasonal naive baselines. Evaluation is strictly chronological: five expanding-window folds on January–November are used for every decision, and December is predicted once as the test set. Cross-validation reveals that tree models fail during the July heat wave because they cannot predict above the training range; learning the deviation from yesterday's load fixes this. Daily lags are the most useful feature group, while weekly and rolling features hurt. The SVR selected on cross-validation reaches a December RMSE of 1,625 kW and MAPE of 1.78%, 50.7% better than the "tomorrow equals today" baseline. A leakage experiment shows that shuffled splits and ten-minute horizons, common in public notebooks, inflate R^2 to 0.999 with the same model. Results assume measured target-hour weather and cover a single year.

Keywords: electricity demand; Tetouan City; time-series validation; support vector regression; feature engineering; data leakage.

Contents

1 Introduction **1**

2 Related Work **1**

3 Dataset and Exploratory Data Analysis **2**

3.1 Data and quality checks 2

3.2 Seasonal pattern 3

3.3 Daily and weekly cycles 4

3.4 Ramadan and holidays 5

3.5 Weather and autocorrelation 6

3.6 From insights to modelling decisions 7

4 Methodology **8**

4.1 Features 8

4.2 Chronological split and cross-validation 9

4.3 Baselines and models 10

4.4 Relative target 10

4.5 Metrics and selection rule 10

5 Baselines and Cross-Validation **11**

5.1 Level 0 and Level 1: one validation month 11

5.2 Level 2: five temporal folds 11

6 Hyperparameter Tuning **12**

7 Ablation and Forecast Horizon **14**

7.1 Feature ablation (RQ2) 14

7.2 Forecast horizon: 1 hour versus 24 hours 15

8 Final Test and Error Analysis **15**

8.1 December test 15

8.2 Error analysis 16

8.3 Accuracy and cost (RQ3) 17

9 Leakage Experiment **18**

10 Conclusion **19**

References **19**

A Supplementary Material **21**

A.1 Reproducibility 21
A.2 Cross-validation folds and Level 0–2 results 21
A.3 Hyperparameter tuning 23
A.4 Ablation 24
A.5 Final test and leakage experiment 24

1 Introduction

Electricity distributors plan purchases and dispatch one day ahead, so they need a forecast of every hour of the next day. This project forecasts the hourly electricity load of Tetouan City (Morocco), using the UCI Power Consumption of Tetouan City dataset[1]. The target is the city total, the sum of the three distribution-zone loads, averaged per hour.

Main problem (`h24`, day-ahead). At 00:00 of day D the model forecasts the 24 hourly loads of day D. The most recent known load is that of 23:00 on day $D-1$, so every load-history feature must lag the target by at least 24 hours.

Secondary problem (`h1`). The same target forecast one hour ahead. It is used only to show how much the forecast horizon changes the difficulty.

The load has a very strong daily cycle, so "tomorrow equals today" is already a strong forecast. A model is useful only if it beats such a seasonal naive baseline. Moreover, ten-minute records are strongly correlated: a randomly shuffled split lets a model interpolate between neighbouring points and report very high accuracy without forecasting anything. The project therefore uses a strictly chronological evaluation. December is reserved as the test set and used once, after all choices have been fixed on January–November.

The study answers three research questions:

RQ1 How much do conventional ML models (linear regression, decision tree, random forest, SVR and XGBoost) improve on seasonal naive baselines under chronological evaluation?

RQ2 How much does each feature group (weather, calendar, special days, load history) contribute?

RQ3 How do hyperparameter tuning and the target formulation affect accuracy and computational cost?

Two supplementary experiments compare the `h24` and `h1` horizons, and measure how common evaluation shortcuts (shuffled splits, very short horizons) inflate reported accuracy.

The contributions are: (i) a chronological pipeline with baselines, five expanding-window folds and a single December test; (ii) a lag-relative target that removes the tree models' failure to extrapolate during the July heat wave; (iii) a feature ablation; and (iv) a leakage experiment that explains why accuracies reported online are higher but not comparable.

2 Related Work

Baselines and models. A seasonal naive forecast repeats the load of the same hour one day or one week earlier; it measures how much information the demand cycle already contains. Regression models map calendar, weather and past-load predictors to the load: linear regression is a low-complexity reference, decision trees learn threshold interactions, random forests average randomised trees, XGBoost adds sequentially fitted trees[2], and RBF support vector regression offers a smooth nonlinear alternative.

Tetouan studies. Table 1 summarises six studies on this dataset, as reported in the course literature review. Salam and El Hibaoui[3] give the original conventional-ML comparison; later work studies optimised regression trees[4], recurrent networks with a December holdout[5], Prophet-based models[6], lag-enriched neural networks[7] and a multi-model Darts comparison[8]. The studies differ in target (one zone, the mean or the sum of zones), resolution, horizon and split, so their numbers are not a common benchmark and this report makes no ranking against them.

Table 1: Published studies on the Tetouan dataset, as summarised in the course literature review. Results are as reported by each study (RMSE in kW).

Study	Target, data	Model	Split	Reported result
Salam & El Hibaoui (2018)[3]	Unclear, 10-min	Random forest	random 75/25	RMSE 4,481
RoSe et al. (2023)[4]	Total, 10-min	Optimised regression tree	70/30, order unclear	R^2 0.976
da Silva & Meneses (2023)[5]	Each zone (several datasets)	LSTM, BiLSTM	Jan–Nov train, Dec test	not reported
Islam et al. (2025)[6]	Zones and their mean	Prophet, NeuralProphet	chronological 75/15/10	MAPE 6.3%
Alsalem (2025)[7]	Zone 3, 10-min	MLP + fuzzy clustering	ordered 70/30	R^2 0.989
Biswas et al. (2026)[8]	Mean of zones, hourly	Random forest (Darts)	chronological	MAPE 1.04%

Validation and leakage. Time-series validation keeps training data before validation data; the expanding-window scheme used here follows `TimeSeriesSplit`[9]. Random search samples a fixed budget of hyperparameter configurations[10]. Leakage means using information that would not be available when the forecast is issued, for example fitting a scaler on test data or using same-time zone loads as inputs. Keeping fitted transformations inside a pipeline avoids the former[11].

Public projects. Ten public GitHub projects on this dataset were surveyed by reading their code (Table 2). Seven shuffle the data before splitting, many use `month` or `dayofyear`, one uses the other zones at the same timestamp as inputs, and none addresses day-ahead forecasting. Section 9 measures how such choices change the reported accuracy.

Table 2: Survey of ten public GitHub projects on the Tetouan dataset (T11), read from their code. Results are as reported by each project and use different targets and splits.

Repository	Target	Data	Split	Load lags	Reported result
Kevinbui16	Total	10-min	random 75/25	no	RMSE 2,624 kW
niharkaria	Total	10-min	random 80/20	no	R^2 0.986
nayana142	Zone 3	10-min	random 80/20	no	R^2 0.956[a]
Merci93	Each zone	10-min	random 70/30	no	R^2 0.82–0.91
SoukainaBoufousse	Each zone	10-min	random 90/10	no	RMSE 353 kW (Zone 3)
ThiagoLevis	Mean of zones	10-min	random 70/30	no	R^2 0.19–0.29
ruwindarowel	Total	10-min	random (K-fold)	no	RMSE 9,907 kW
atishk28	Total	10-min	chronological 80/20	no	R^2 0.834
daniahfarooq	Each zone	hourly	chronological 80/20	1 h, 24 h	R^2 0.973 (Zone 1)
AtharvaPandey-coder	Zone 1	10-min	chronological 80/20	last 24 h	R^2 0.971

[a] Uses the other zones at the same timestamp as inputs (leakage).

3 Dataset and Exploratory Data Analysis

3.1 Data and quality checks

The dataset[1] contains 52,416 ten-minute records from 1 January 2017 00:00 to 30 December 2017 23:50 (364 days): temperature, humidity, wind speed, two diffuse-flow (solar radiation) measurements and the loads of three distribution zones (Quads, Smir, Boussafou). Averaging each hour gives 8,736 hourly rows. The target is the hourly mean of the three-zone sum, in kW (Table 3).

Data-quality checks find no missing values, duplicates, irregular time steps or non-positive values, so no row is removed or imputed. Two anomalies are kept as they are. On 20 April at noon, load moves from Zone 1 to Zone 2 for a few hours; the city total barely changes, which is one reason to forecast the total. Wind speed is almost binary (57% of samples ≤ 0.1, 38.6% between 4.8 and 5.1), which suggests an on/off instrument signal; it is kept and listed as a limitation. Unless stated otherwise, the figures below use January–November only, so that the December test set does not influence feature design.

Table 3: Whole-year descriptive statistics of the original ten-minute observations (T01). Blank source units are left unspecified.

Variable	Unit	Mean	SD	Minimum	Maximum
Temperature	°C	18.81	5.82	3.25	40.01
Humidity	%	68.26	15.55	11.34	94.80
WindSpeed	–	1.96	2.35	0.05	6.48
GenDiffuse	–	182.70	264.40	0.00	1,163.00
Diffuse	–	75.03	124.21	0.01	936.00
Z1	kW	32,344.97	7,130.56	13,895.70	52,204.40
Z2	kW	21,042.51	5,201.47	8,560.08	37,408.86
Z3	kW	17,835.41	6,622.17	5,935.17	47,598.33
Total	kW	71,222.89	17,143.14	36,785.04	134,208.15

3.2 Seasonal pattern

Figure 1 shows a flat load of about 66,000–70,000 kW from January to May, a summer rise to a July peak of 88,174 kW (about 30% above the base level), and a quick fall in September. December, the test month, averages 63,751 kW, close to November (65,105 kW). The summer peak matters later: models trained on the first half of the year must forecast July loads higher than anything seen in training.

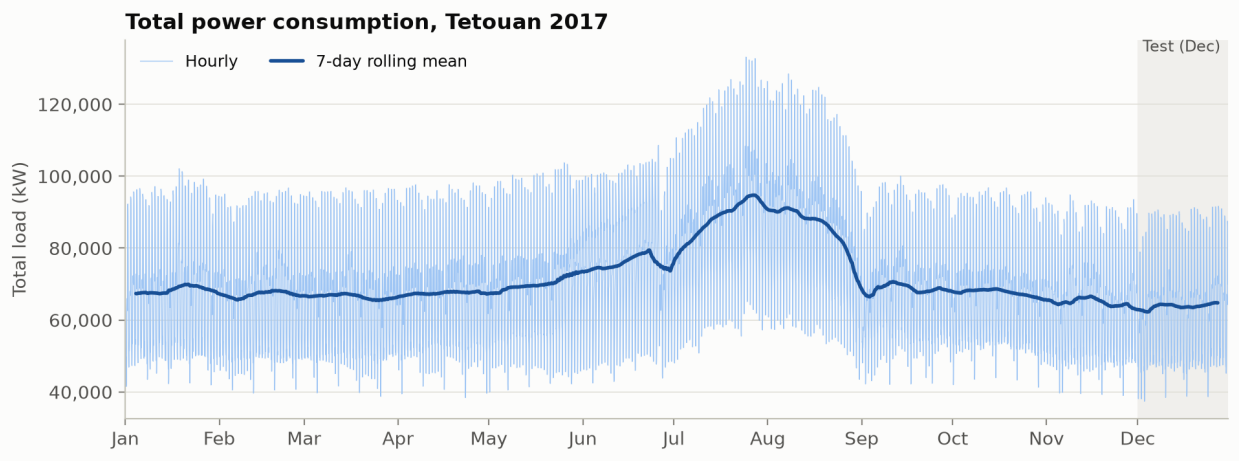


Figure 1: Hourly total load over 2017 with a centred seven-day mean (F01). December (test) is shaded.

The three zones behave differently (Figure 2). Zone 3 rises most in summer (its July mean is 2.6 times its December mean), and from September Zone 2 grows while Zone 3 falls, consistent with load transfers between networks. Forecasting the city total avoids these transfers.

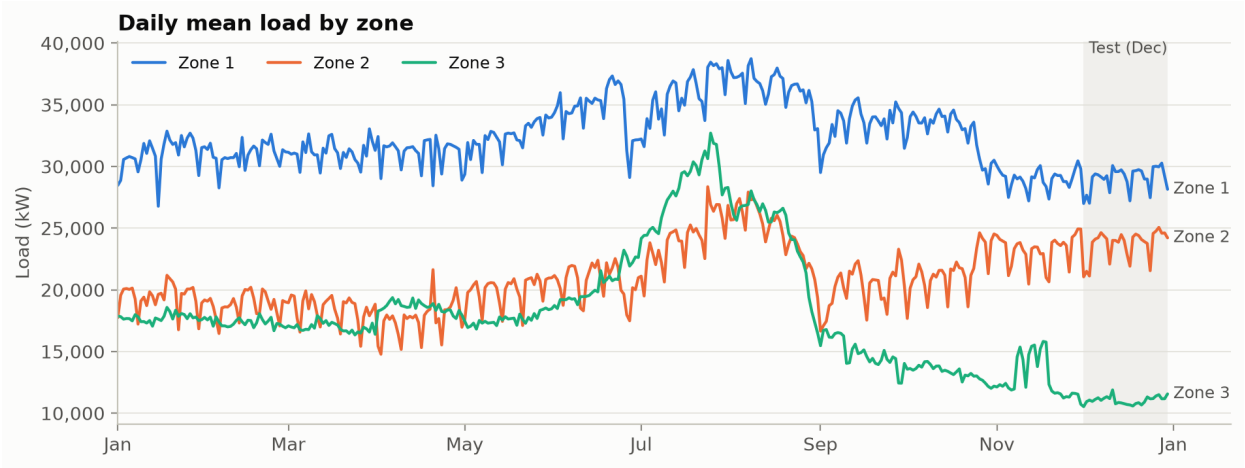


Figure 2: Daily mean load of the three distribution zones (F02).

3.3 Daily and weekly cycles

Every season has the same daily shape (Figure 3): a minimum at 06:00–07:00, a daytime plateau and an evening peak at 19:00–20:00, about twice the minimum. The daily cycle is the strongest structure in the data and motivates the hour features and the daily lags. Summer is higher at every hour, most visibly at night.

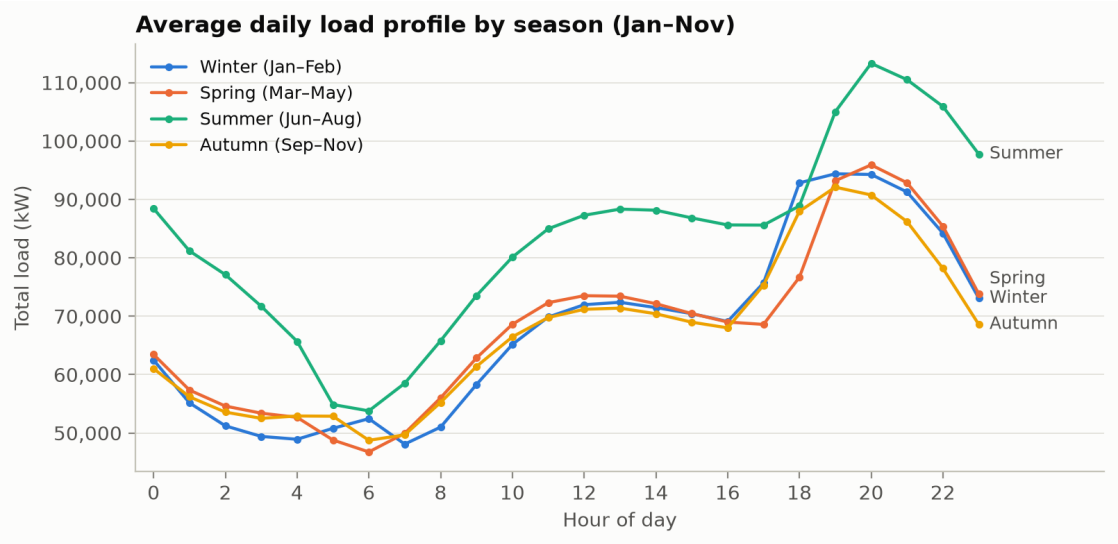


Figure 3: Mean daily load profile by season, January–November (F03).

The right panel of Figure 4 removes the daily cycle by comparing each hour with the Monday–Friday mean of the same hour. Sundays are 5.9% lower on average and about 15% lower at 07:00–08:00; Saturday mornings are 4–6% lower. A "copy yesterday" forecast will therefore be too low on Mondays and too high on Sundays, which the baseline results confirm (Section 5).

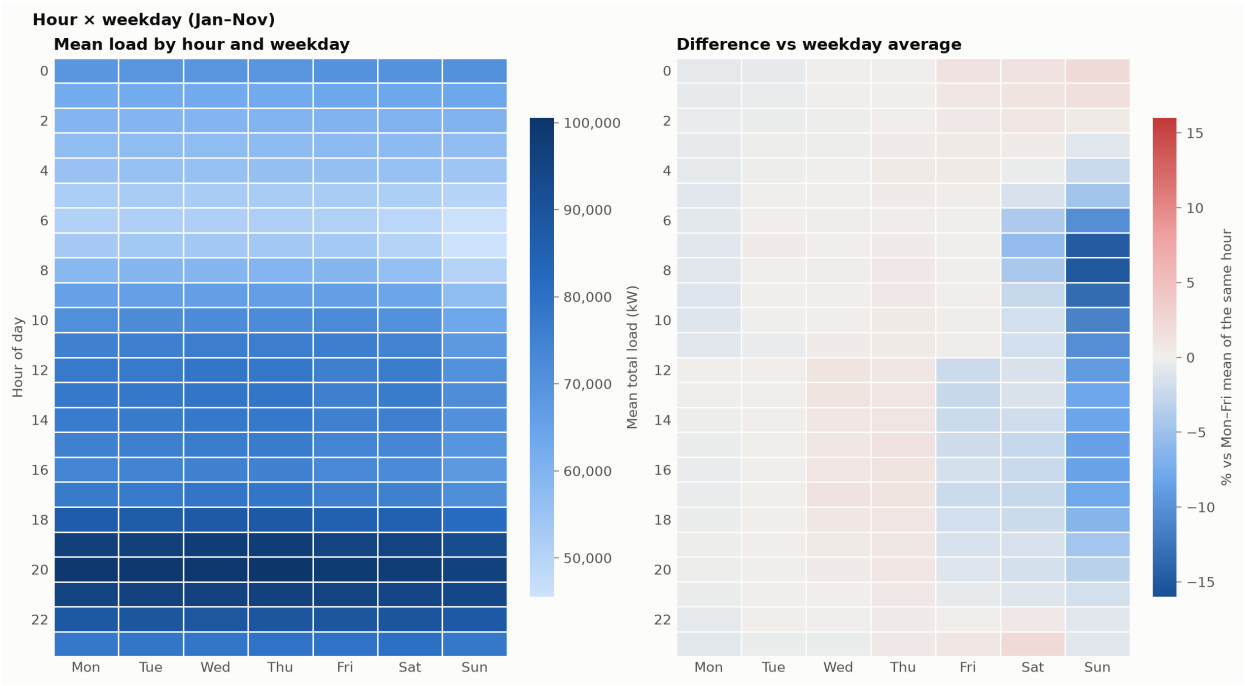


Figure 4: Mean load by hour and weekday (left) and deviation from the Monday–Friday mean of the same hour (right), January–November (F04).

3.4 Ramadan and holidays

Ramadan (27 May – 25 June) reshapes the daily profile (Figure 5): the load between 00:00 and 03:00 is up to 32,142 kW higher than in the four preceding weeks (the pre-dawn meal), mornings are lower and evenings higher. Mean temperature is only 1.8 °C higher, so weather does not explain the change; this motivates an `is_ramadan` flag.

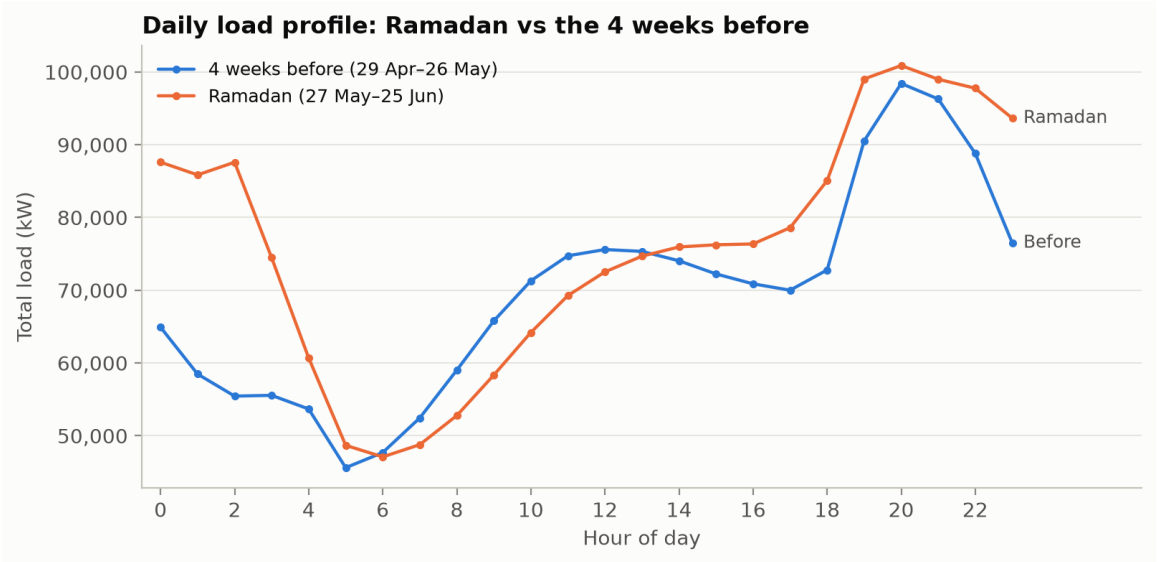


Figure 5: Hourly load profile during Ramadan and the four preceding weeks, holidays excluded (F05).

Figure 6 compares each day with the median of the surrounding 15 days, which removes the seasonal trend. Sundays are 5.9% lower. Religious holidays drop sharply (Eid al-Fitr and Eid al-Adha: −8% to −13%, mean about −9%), whereas civil holidays are weaker and uneven (mean about −3%; 1 May and 6 November about −6%, other dates close to normal days). The holiday flag is therefore split into religious and civil holidays.

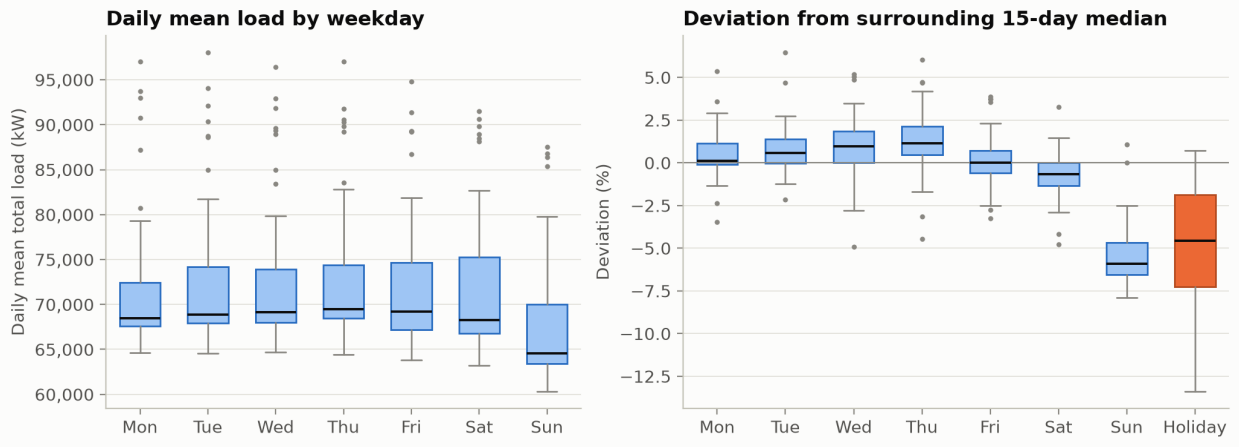


Figure 6: Daily load by weekday and on holidays, as deviation from the centred 15-day median, January–November (F08).

3.5 Weather and autocorrelation

Temperature is the weather variable most correlated with the load ($r = 0.49$; humidity -0.30, wind speed 0.21, radiation 0.15 and 0.02). The relation is nonlinear (Figure 7): the load barely depends on temperature below about 22 °C and rises quickly above it (cooling). The three time-of-day bands show that the hour explains more than temperature, which favours nonlinear models. The zones are strongly correlated with the total (Figure 8) but cannot be inputs at the same timestamp, since they are components of the target.

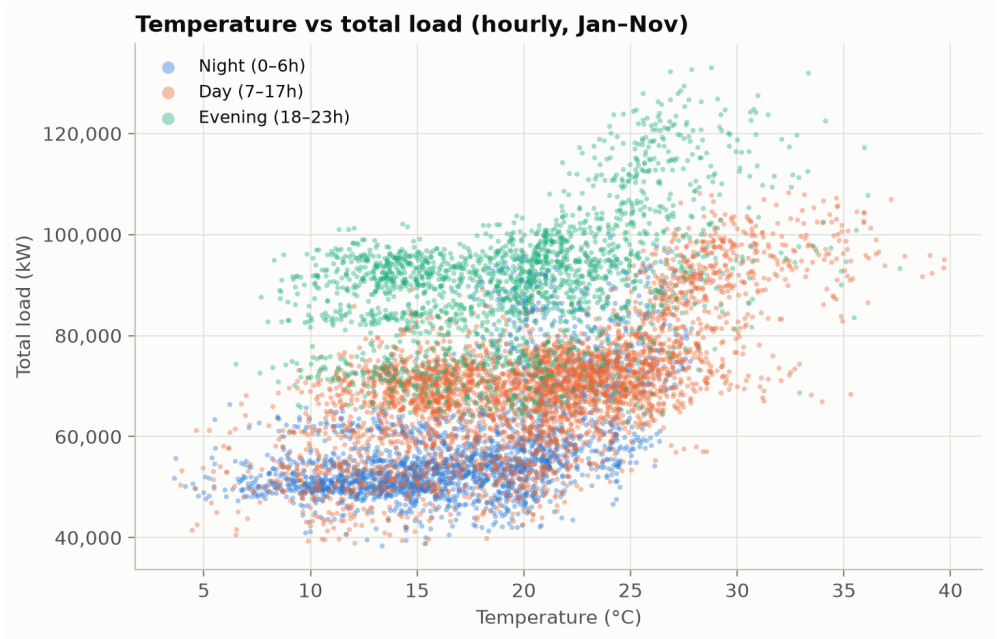


Figure 7: Temperature against total load, coloured by time of day, January–November (F06a).

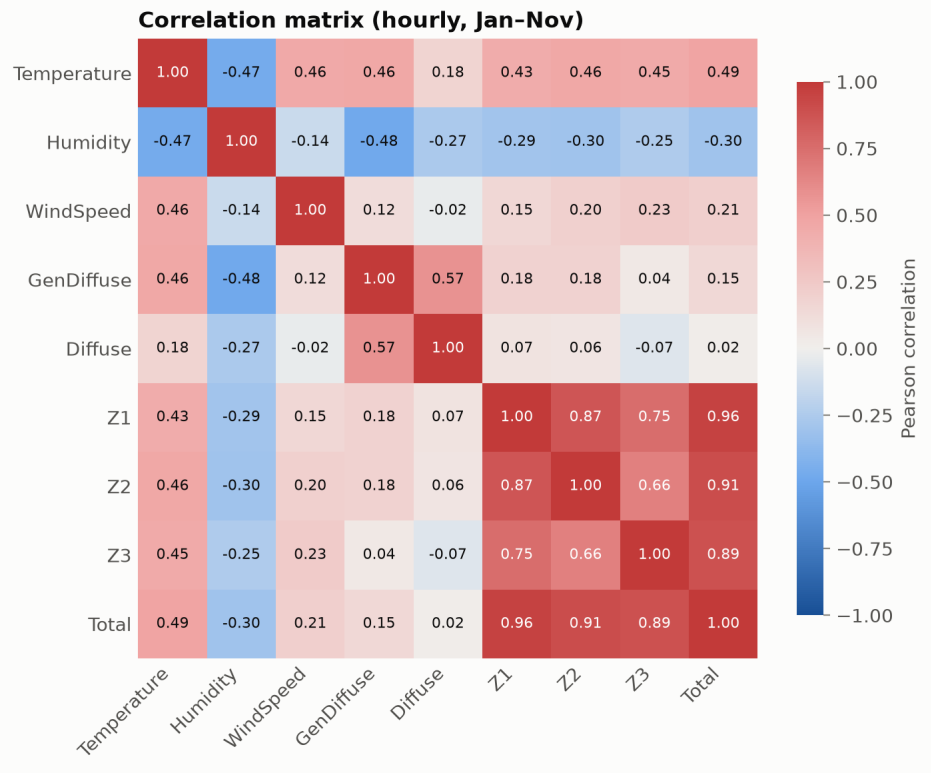


Figure 8: Pearson correlations between hourly variables, January–November (F06b).

The autocorrelation function (Figure 9) peaks every 24 hours: 0.98 at lag 24, slowly decaying to 0.95 at lag 72; it is negative (−0.23) at lag 12, when day meets night. The value at lag 168 (0.95) stands above its neighbours (0.88 at lags 167 and 169), revealing a weekly cycle. These peaks justify `lag_24`, `lag_48` and the tested `lag_168` for `h24`, and `lag_1`–`lag_3` for `h1`. They also mean that Naive-24h is a hard baseline to beat.

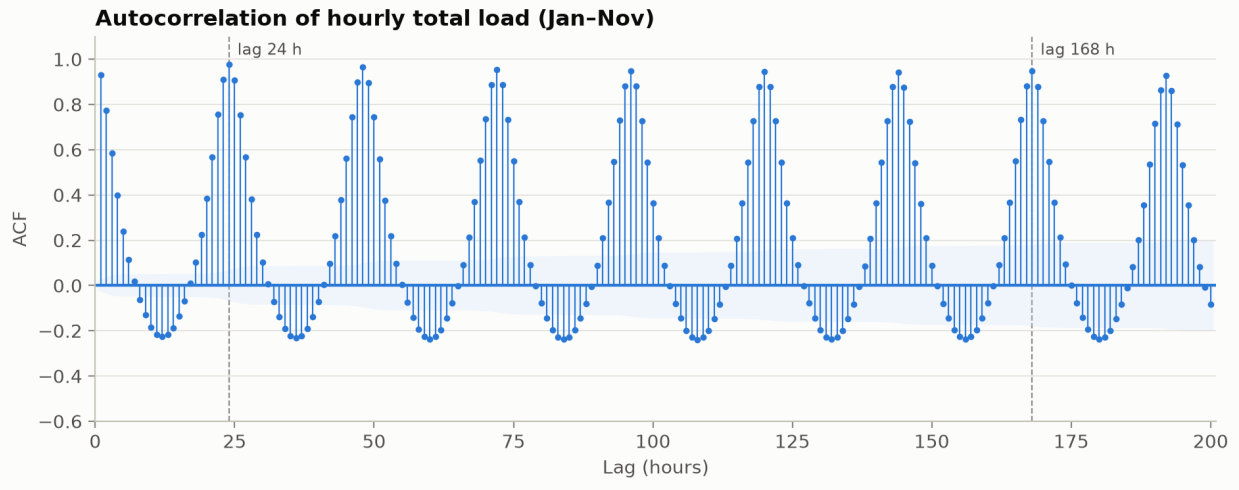


Figure 9: Autocorrelation of the hourly total load up to lag 200, January–November (F07).

3.6 From insights to modelling decisions

Table 4 links each EDA finding to the way it is used in training, and to the later result that checks whether the decision was right.

Table 4: EDA insights, how they are used in training, and where they are checked.

Insight (evidence)	Use in training	Checked by
Clean data, no gaps (quality checks)	No row removed or imputed; only hourly averaging	–
Zones exchange load; the total is stable (Figure 2)	Target is the city total; zone loads are never inputs (they are parts of the target)	–
Strong daily cycle, ACF 0.98 at lag 24 (Figures 3 and 9)	Hour and its sin/cos; `lag_24`, `lag_48`; Naive-24h as the baseline to beat; relative target $y_t - y_{t-24}$	Ablation: daily lags cut RMSE by 67% (Section 7)
Weekly cycle: Sundays −6%, Monday mornings unlike Sundays; ACF peak at lag 168 (Figures 4 and 9)	Weekday, weekend flag and their sin/cos; Naive-168h baseline; `lag_168` tested in the ablation	Naive-24h errs most on Mondays (Section 5); the final model cuts the Monday error by 81% (Section 8)
Summer loads 30% above the base level (Figure 1)	Folds start in July so the peak is evaluated; no `month`/`dayofyear`; relative target for tree models	July fold fails for absolute trees (Section 5.2); fixed by the relative target (Section 6)
Ramadan reshapes the night profile, not explained by weather (Figure 5)	`is_ramadan` flag	Special days cut RMSE by 9% (Section 7)
Religious holidays −9%, civil holidays −3% and uneven (Figure 6)	Separate religious and civil holiday flags	Mawlid is the worst test day (Section 8)
Temperature effect nonlinear above 22 °C and depends on the hour (Figure 7)	Weather of the target hour as input; nonlinear models (DT, RF, SVR, XGB) next to linear regression	Removing weather changes RMSE by at most 5% (Section 7)
Neighbouring hours are almost identical (high ACF at short lags)	Chronological split and `TimeSeriesSplit`, never shuffled	Shuffled splits inflate R^2 to 0.999 (Section 9)
Wind speed is almost binary	Kept as is	Listed as a limitation (Section 10)

4 Methodology

4.1 Features

All features are built on the full chronological series and the data are split afterwards; every load-history feature is shifted by at least the horizon h, so this order cannot leak future values. Table 5 lists the groups.

Calendar. Hour, weekday and a weekend flag, plus sine/cosine encodings so that 23:00 and 00:00, or Sunday and Monday, are close. No `month` or `dayofyear` feature is used: December never appears in training, so such a feature would force the model to extrapolate.

Special days. Separate flags for religious holidays, civil holidays and Ramadan (Section 3).

Load history. Lags y_{t-k} with $k \geq h$: `lag_24` and `lag_48` for `h24`; `lag_1`, `lag_2`, `lag_3` and `lag_24` for `h1`. Rolling means and the rolling standard deviation are computed on the series shifted by h first, so no window contains y_t. A checker recomputes every lag and rolling value by hand at 200 random timestamps, and a deliberately unshifted rolling mean is correctly detected.

Weather. The five measured weather variables of the target hour. This is a perfect-forecast assumption: in operation they would have to be replaced by weather forecasts (Section 10).

The models use feature set D (17 columns for `h24`, 19 for `h1`). The weekly lag and rolling features

are built but only used by the Naive-168h baseline and the ablation, which showed that they hurt (Section 7). The first 191 hours (168 for `h1`) lack history and are dropped, so the data start on 8 January.

Table 5: Feature groups (T03). Model inputs are feature set D.

Group	Features	Used by the models
Weather (5)	Temperature, humidity, wind speed, general and diffuse flows (target hour)	both horizons
Calendar (7)	Hour, weekday, weekend flag, sin/cos of hour and weekday	both horizons
Special days (3)	Religious holiday, civil holiday, Ramadan	both horizons
Daily lags (2)	y_{t-24}, y_{t-48}	`h24`
Recent lags (4)	y_{t-1}, y_{t-2}, y_{t-3}, y_{t-24}	`h1`
Weekly / rolling (4)	y_{t-168}, 24-h rolling mean and SD, 168-h rolling mean (all shifted by h)	no (ablation only)

4.2 Chronological split and cross-validation

The data are split by date (Table 6): training January–October, validation November, and test December. Development (January–November) is used for every decision: features, hyperparameters and model selection. December is predicted once, at the final step.

Table 6: Chronological partitions after history-related row removal (T02). All dates are in 2017; end times are 23:00.

Setting	Partition	Start	End	Hours
h24	Train	01-08 23:00	10-31 23:00	7,105
h24	Validation	11-01 00:00	11-30 23:00	720
h24	Development	01-08 23:00	11-30 23:00	7,825
h24	Test	12-01 00:00	12-30 23:00	720
h1	Train	01-08 00:00	10-31 23:00	7,128
h1	Validation	11-01 00:00	11-30 23:00	720
h1	Development	01-08 00:00	11-30 23:00	7,848
h1	Test	12-01 00:00	12-30 23:00	720

Within development, `TimeSeriesSplit(n_splits=5, test_size=720)` creates five expanding-window folds, each validating the 30 days that follow its training window (Figure 10; exact dates in Table 12, appendix). Validation windows run from early July to the end of November, so the summer peak is part of the evaluation. The last fold coincides with the November validation month. Scalers are fitted inside each training fold through sklearn pipelines.

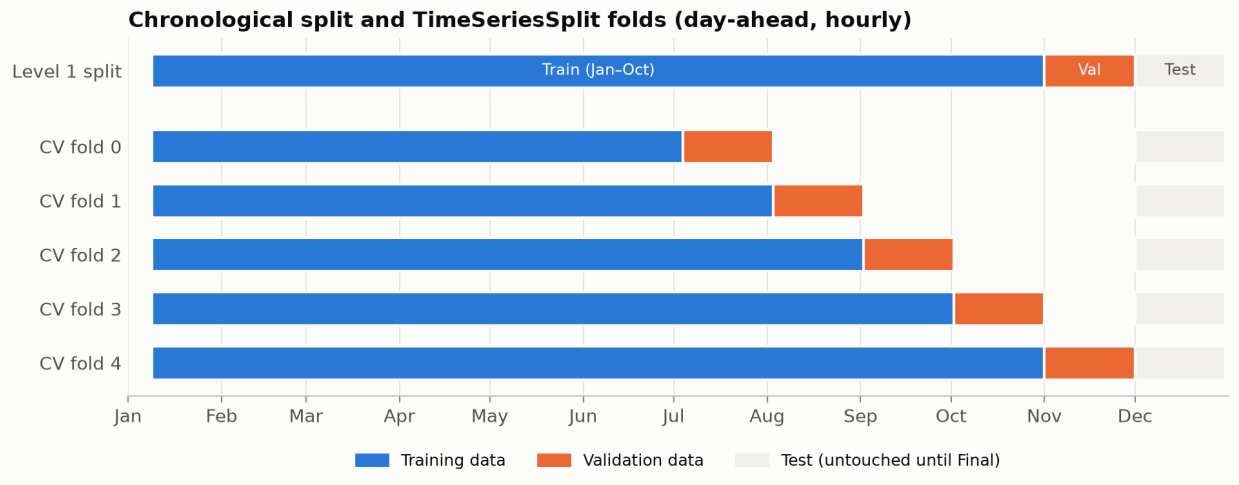


Figure 10: Chronological split and the five expanding-window folds (F10). December is outside every fold.

4.3 Baselines and models

Two seasonal naive baselines set the minimum level a model must beat: **Naive-24h**, $\widehat{y}_t = y_{t-24}$ ("tomorrow equals today"), and **Naive-168h**, $\widehat{y}_t = y_{t-168}$ (same hour last week). The five models are linear regression (Ridge when tuned), a decision tree (DT), a random forest (RF, 100 trees by default), RBF SVR and XGBoost (XGB). LR and SVR standardise their inputs; SVR also standardises the target, because the default $\epsilon = 0.1$ is meaningless for loads around 70,000 kW. All random seeds are 42.

4.4 Relative target

Tree-based models predict the mean of the training targets in a leaf, so they can never predict above the highest load seen in training. In the July fold, 12% of the validation hours exceed the training maximum (133,130 versus 110,711 kW), and absolute tree models under-forecast them (Section 5.2). The relative target asks the model to learn the deviation from yesterday instead:

$$r_t = y_t - y_{t-24}, \qquad \widehat{y}_t = y_{t-24} + g(\mathbf{x}_t), \tag{1}$$

where g is any regressor and y_{t-24} also remains an input. The load level may reach new highs, but the day-to-day deviation stays within the range seen in training; the level is carried by y_{t-24} through an addition, which has no upper bound. Relative versions are marked with Δ (e.g. RF Δ). Both target types are tuned for every model and the better one is chosen on CV.

4.5 Metrics and selection rule

Models are compared with RMSE and MAE (kW), MAPE (%) and R^2. CV results are the mean and sample standard deviation over the five folds. The skill score against the baseline is

$$S = 1 - \frac{\mathrm{RMSE}_{\mathrm{model}}}{\mathrm{RMSE}_{\mathrm{Naive\text{-}24h}}}, \tag{2}$$

the share of the baseline error that the model removes. The final model is the tuned version with the lowest mean CV RMSE; the test results are reported but never used for selection.

5 Baselines and Cross-Validation

5.1 Level 0 and Level 1: one validation month

The first experiment checks whether the models can beat the naive baselines before any tuning. The five models are trained with default hyperparameters on January–October and evaluated on November (Figure 11; full numbers in Table 13, appendix). Naive-24h reaches RMSE 3,076 kW, MAPE 3.27% and $R^2 = 0.951$: copying yesterday already explains most of the variance, as the ACF predicted. Naive-168h is slightly worse (3,203 kW).

Every model except DT clearly beats Naive-24h: XGB has the lowest RMSE (2,161 kW, skill 29.7%), followed by SVR (2,195), RF (2,228) and LR (2,289). The default DT grows until each leaf holds a single sample: its training error is zero, but its validation RMSE (2,992 kW) is barely below the baseline. The guideline checkpoint is passed: the models beat the naive baseline before tuning.

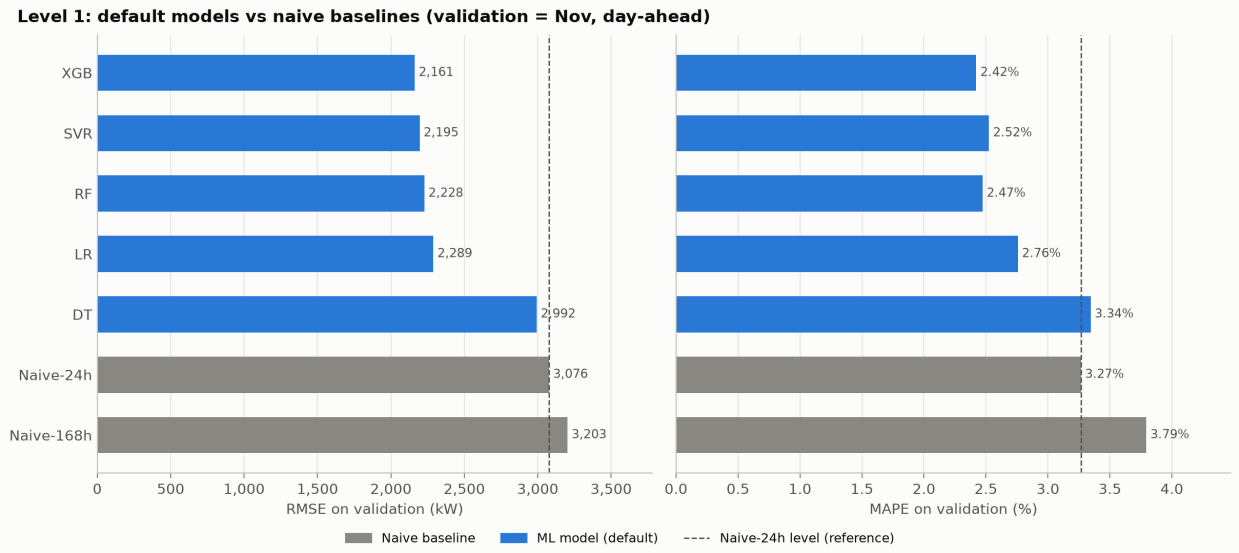


Figure 11: Level 1: November RMSE and MAPE of the default models and the baselines. The dashed line marks Naive-24h.

The weekday effect seen in the EDA shows up in the errors: Naive-24h has an MAE of 4,077 kW on Mondays (it copies Sunday) and 3,466 kW on Sundays (it copies Saturday), 2.5–3 times its Tuesday–Friday level. The best model roughly halves the Monday error thanks to the weekday features.

5.2 Level 2: five temporal folds

One validation month may be unrepresentative. The second experiment keeps the same default models and only replaces the November split with the five temporal folds (Figure 12; numbers in Tables 14 and 15, appendix). The ranking changes: LR becomes the best default model (2,792 ± 623 kW), followed by RF (3,070) and XGB (3,265), while SVR (3,969) and DT (4,048) fall behind Naive-24h (3,392). The last fold reproduces the November results exactly, so a single month would have given a misleading ranking.

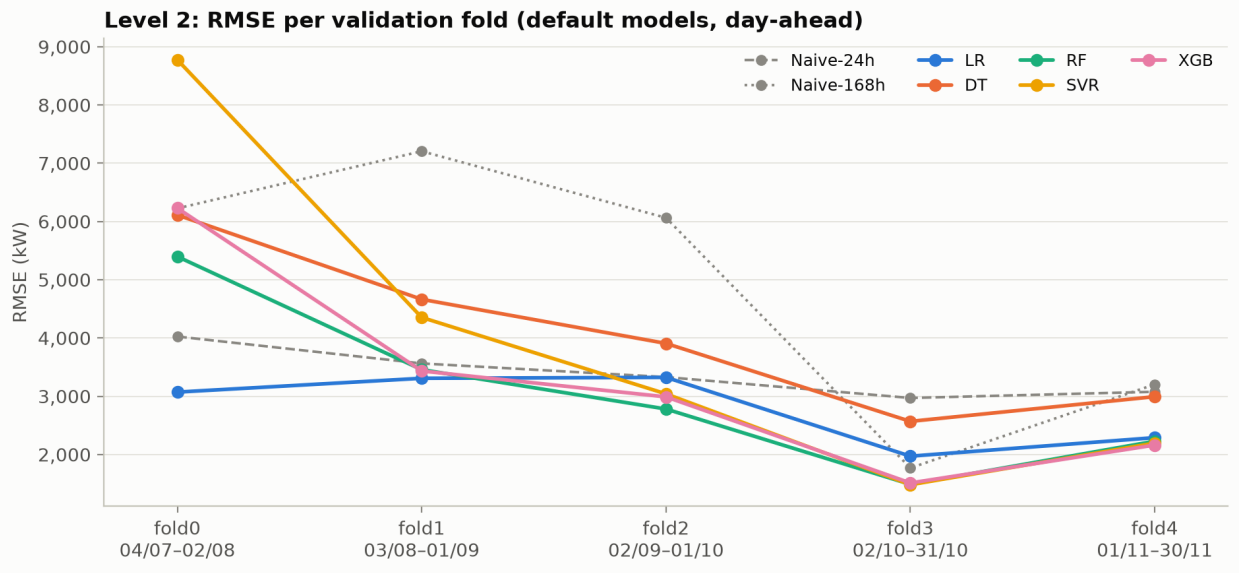


Figure 12: Level 2: RMSE of the default models and the baselines in each validation fold. Fold 0 covers 4 July – 2 August.

The large differences come from fold 0 (July). There, RF, XGB, SVR and DT reach 5,397, 6,230, 8,775 and 6,111 kW, against 3,071 kW for LR and 4,024 kW for Naive-24h. The cause is extrapolation: 12% of the July hours exceed the highest load in the training window, and tree models cannot predict above that range (Section 4.4); the RBF SVR also degrades far from its training data. From fold 2 onwards, RF and XGB beat LR again. This finding motivates the relative target tested in the next section.

6 Hyperparameter Tuning

This experiment tunes each model with `RandomizedSearchCV` on the same five folds as Level 2, so the change from Level 2 to Level 3 is the effect of tuning. Random search samples a fixed number of configurations from given ranges[10]: 20 for Ridge, 40 for DT, 30 for RF and SVR, and 50 for XGB (Table 7). Each model is tuned twice, with the absolute and with the relative target, trying the same configurations, and the one with the lowest mean CV RMSE is kept. Linear regression is tuned as Ridge, because plain least squares has no hyperparameter. Log-uniform distributions are used for parameters whose scale is unknown, so that every order of magnitude is equally likely to be tried.

Table 7: Hyperparameter search spaces (T06). Each model is tuned twice (absolute and relative target) with the same space; *Candidates* is the number of sampled configurations.

Model	Candidates	Hyperparameter	Values / distribution
LR	20	`alpha`	log-uniform(0.001, 1000.0)
DT	40	`max_depth`	4, 6, 8, 10, 15, 20, None
		`min_samples_leaf`	1, 5, 10, 20, 50
		`min_samples_split`	2, 10, 20, 50
		`max_features`	None, sqrt, 0.5, 0.8
RF	30	`n_estimators`	100, 200, 300, 500
		`max_depth`	10, 20, 30, None
		`min_samples_leaf`	1, 2, 5, 10
		`max_features`	sqrt, 0.3, 0.5, 0.8, 1.0
SVR	30	`C`	log-uniform(0.1, 1000.0)
		`gamma`	log-uniform(0.0001, 1)
		`epsilon`	0.01, 0.05, 0.1, 0.2
XGB	50	`n_estimators`	integer 200 ... 1499
		`learning_rate`	log-uniform(0.01, 0.2)
		`max_depth`	integer 3 ... 10
		`subsample`	uniform(0.6, 1.0)
		`colsample_bytree`	uniform(0.6, 1.0)
		`min_child_weight`	integer 1 ... 10
		`reg_lambda`	log-uniform(0.1, 10.0)

Figure 13 compares, for each model, the default version with the two tuned versions (numbers in Table 16, appendix). The best version is the absolute SVR: 2,411 ± 766 kW, 39% below its default. Tuning matters most for SVR, which is very sensitive to its regularisation parameter C, and hardly at all for Ridge. The relative SVR (2,427), relative RF (2,429) and relative XGB (2,441) follow within 1.3%, less than the fold-to-fold variation.

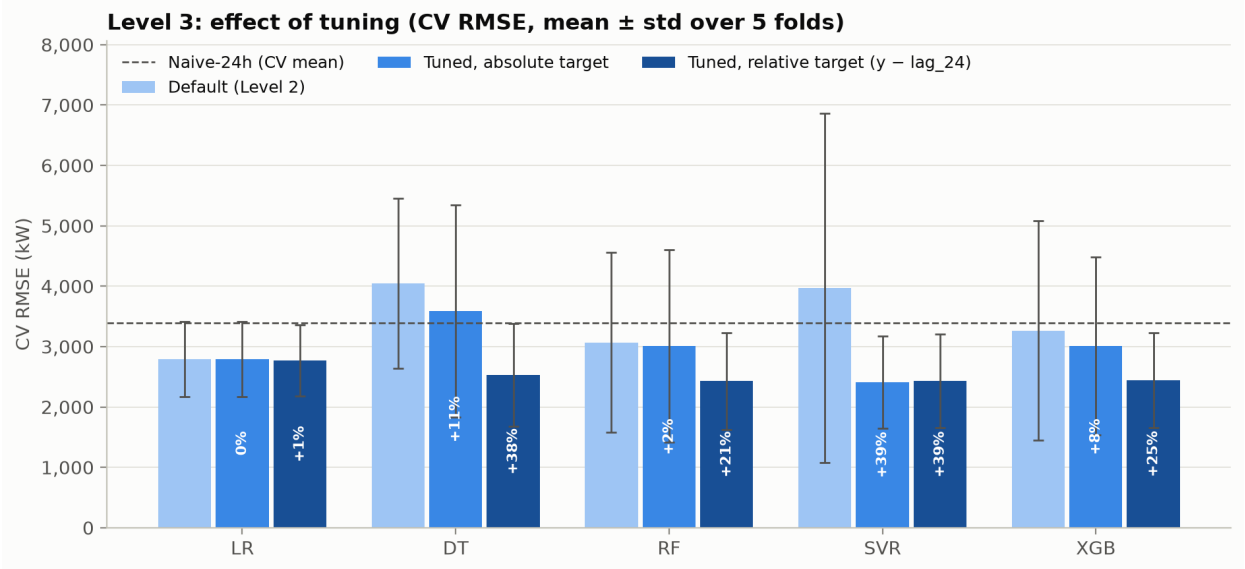


Figure 13: CV RMSE per model: default, tuned with the absolute target and tuned with the relative target (mean ± SD over five folds). Percentages compare with the default model.

For the tree models the relative target matters more than tuning. Tuning alone lowers absolute RF

by only 2% and absolute XGB by 8%, while the relative target brings them to 2,429 and 2,441 kW. The gain comes from the July fold, where RMSE drops from 5,554 to 3,215 kW for RF and from 5,344 to 2,979 kW for XGB. The relative target is better for LR, DT, RF and XGB; for SVR the two versions are almost tied.

The same folds are used to choose and to report the configurations, so these CV scores are slightly optimistic; the December test measures the selected model again.

7 Ablation and Forecast Horizon

7.1 Feature ablation (RQ2)

To measure what each feature group contributes, the groups are added one at a time and RF and XGB are cross-validated after each step, with the tuned hyperparameters kept fixed so that only the features change. Set A contains weather only; B adds calendar; C adds special days; D adds the daily lags; E adds the weekly lag and the rolling features; F is E without weather (Figure 14; numbers in Table 18, appendix).

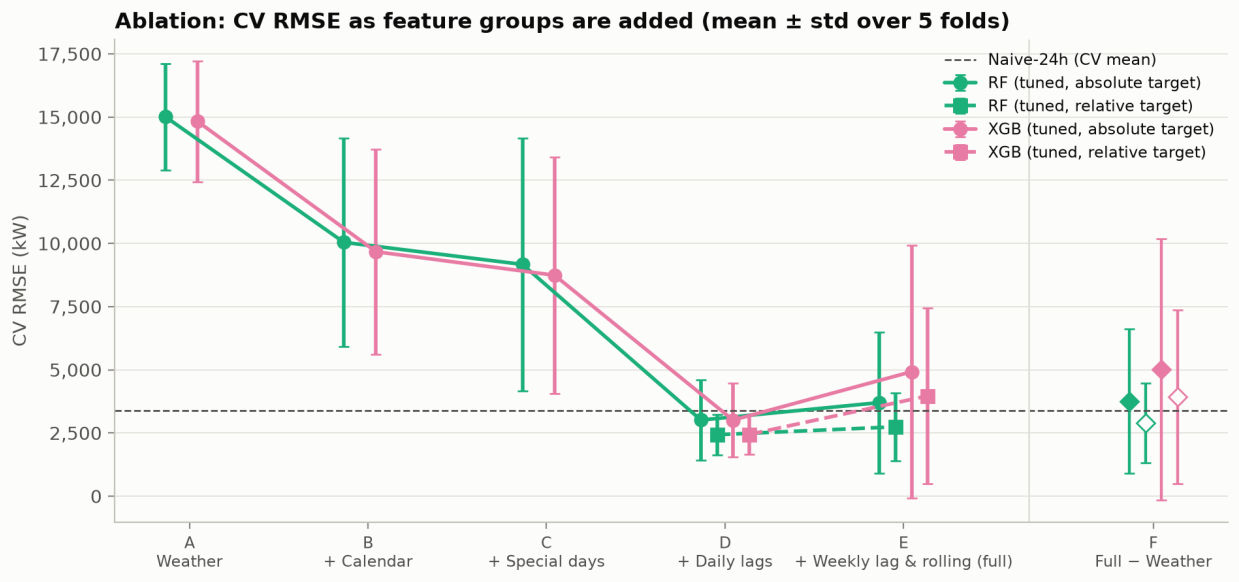


Figure 14: Ablation: CV RMSE as feature groups are added. Dashed lines and squares: relative target; diamonds: set F (E without weather).

Daily lags are by far the most important group: from C to D the RMSE of RF falls from 9,168 to 3,010 kW (−67%), and XGB falls by 66%. Calendar features reduce the error by about a third (A to B) and special days by another 9% (B to C), but without load history the models remain far worse than Naive-24h.

Adding the weekly lag and the rolling means (D to E) makes every variant worse, by 13% to 64%, almost entirely in the July and August folds. The rolling means average out the daily cycle and keep only the seasonal level, which in July lies above every training value for 87–96% of the hours. In the stable autumn folds they help by about 200 kW at most. They are therefore excluded, and set D is the model feature set; the weekly pattern is still represented by the weekday features. Removing weather (E to F) changes the error by at most 5%: weather adds little once daily lags are present.

7.2 Forecast horizon: 1 hour versus 24 hours

For `h1`, every model keeps its `h24` configuration (target type and tuned hyperparameters) and only the features change, adding `lag_1`–`lag_3`. Both horizons use the same five validation windows (Table 8).

Table 8: Horizon comparison on development CV (T10): mean RMSE in kW and MAPE in percent. Each model keeps its h24 target type and hyperparameters at both horizons.

Model	Target	h24 RMSE	h1 RMSE	h24 MAPE (%)	h1 MAPE (%)
Naive-24h	–	3,392.4	3,392.4	3.28	3.28
Naive-168h	–	4,892.6	4,892.6	4.92	4.92
LR	relative	2,770.0	2,509.0	2.87	2.60
DT	relative	2,524.9	2,550.7	2.40	2.43
RF	relative	2,429.3	2,381.1	2.36	2.32
SVR	absolute	2,410.6	1,866.4	2.42	1.86
XGB	relative	2,441.0	2,321.6	2.38	2.28

Forecasting one hour ahead is clearly easier for SVR: its CV RMSE falls from 2,411 to 1,866 kW (-23%) and its MAPE from 2.42% to 1.86%. LR improves by 9% and XGB by 5%, but RF improves by only 2% and DT not at all. These tree models learn the deviation from `lag_24` with hyperparameters tuned for the day-ahead task (for example a depth-4 DT), so they make little use of the latest hour, whereas SVR uses the absolute target and benefits directly. The horizon thus strongly drives difficulty, which is also why accuracies obtained with very short horizons cannot be compared with day-ahead results.

8 Final Test and Error Analysis

8.1 December test

Once every choice is fixed, each model is refitted on the whole development set (7,825 hours, 8 January – 30 November) and predicts the 720 hours of December once. Lag inputs use the loads measured up to the previous day. The final model was chosen before this step: the absolute-target SVR, which has the lowest CV RMSE.

The selected SVR reaches RMSE 1,625 kW, MAE 1,095 kW, MAPE 1.78% and $R^2 = 0.987$ (Table 9): a 50.7% RMSE reduction against Naive-24h (3,296 kW). It also beats Naive-168h (2,620 kW), which happens to be the stronger baseline in December. Every tuned model beats Naive-24h, and the better version of each model also beats Naive-168h. The relative SVR has the lowest test RMSE (1,419 kW), but it is not selected: switching after seeing the test would turn December into a validation set, and the two versions were tied on CV. Test errors are 20–35% below the CV estimates, because December has no extrapolation problem and the final models see more data, whereas the CV folds include the summer peak. The four best versions on CV are all among the five best on the test set, so the CV ranking was reliable.

Table 9: December test, h24 (T08), ordered by test RMSE. The model chosen on CV before the test is SVR-tuned. RMSE/MAE in kW; skill is the RMSE reduction against Naive-24h.

Model	RMSE	MAE	MAPE (%)	R^2	Skill (%)
SVR Δ-tuned	1,419.1	1,046.1	1.72	0.990,0	56.95
SVR-tuned	1,624.6	1,094.6	1.78	0.986,9	50.71
XGB-tuned	1,711.5	1,233.5	2.03	0.985,4	48.08
XGB Δ-tuned	1,719.2	1,209.9	2.00	0.985,3	47.84
RF Δ-tuned	1,761.9	1,171.4	1.93	0.984,6	46.55
XGB-default	1,891.0	1,330.3	2.15	0.982,2	42.63
DT Δ-tuned	2,005.2	1,316.7	2.20	0.980,0	39.17
RF-tuned	2,017.1	1,331.7	2.19	0.979,8	38.81
RF-default	2,044.3	1,360.8	2.24	0.979,2	37.98
SVR-default	2,063.7	1,300.5	2.20	0.978,8	37.39
LR-tuned	2,169.6	1,624.8	2.74	0.976,6	34.18
LR-default	2,169.6	1,624.8	2.74	0.976,6	34.18
LR Δ-tuned	2,217.3	1,629.1	2.74	0.975,5	32.73
Naive-168h	2,619.8	1,775.4	2.87	0.965,8	20.52
DT-tuned	2,846.6	1,960.3	3.19	0.959,7	13.64
DT-default	2,989.6	1,987.4	3.28	0.955,5	9.30
Naive-24h	3,296.3	2,071.5	3.39	0.945,9	0.00

8.2 Error analysis

The forecast follows the actual load for almost the whole month (Figure 15). The MAE equals 1.72% of the mean December load (63,751 kW), half the error of "tomorrow equals today". The errors are centred on zero (bias −202 kW) with no trend over the month (residual plots in Figure 18, appendix).

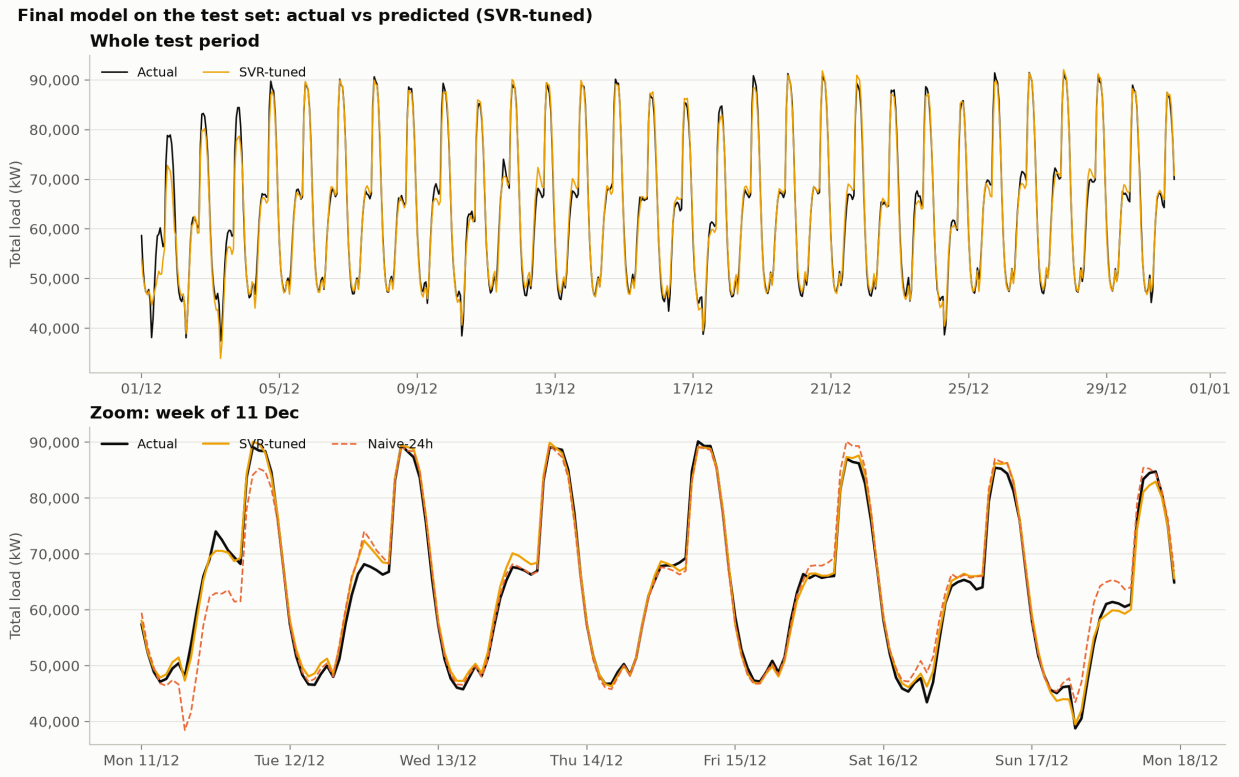


Figure 15: Actual and forecast load of the selected SVR over December, with a one-week zoom including Naive-24h.

By hour and weekday (Figure 16), the model removes most of the baseline error during the day and on Mondays (MAE 1,007 versus 5,294 kW for Naive-24h, which copies Sunday). The worst day by far is 1 December, the religious holiday of Mawlid: MAE 4,579 kW, with the daily mean forecast at 54,957 kW against 58,574 kW actual (Table 19, appendix). The religious-holiday flag was learned mostly from the two Eid holidays (−8% to −13%), so the model predicts too large a drop. Saturday 2 and Sunday 3 December, around the holiday, are also among the five worst days.

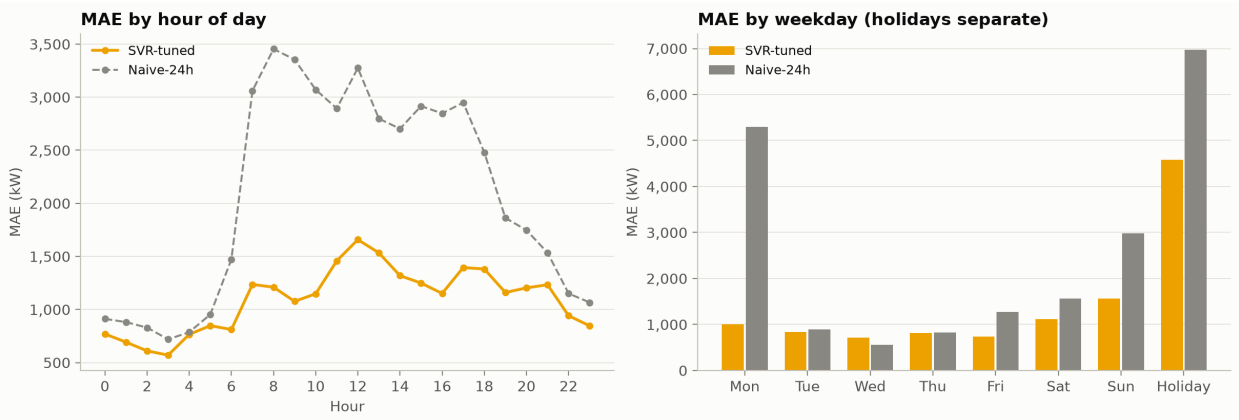


Figure 16: December MAE by hour of day and by weekday (holidays separate) for the selected SVR and Naive-24h.

8.3 Accuracy and cost (RQ3)

SVR is the most accurate model but the slowest to fit (a few seconds); relative XGB and relative RF are 6–8% worse in test RMSE and fit in under half a second; Ridge fits instantly but is about 35% worse (Table 20, appendix). With about 8,000 training rows every model is fast enough for

daily operation, so accuracy should come first.

9 Leakage Experiment

Public projects on this dataset often report $R^2 \approx 0.99$, above the 0.987 of this project. This experiment, run after the final test, explains the gap by evaluating *the same Random Forest* (the tuned absolute RF) under four set-ups that mimic common practice:

- **S1:** ten-minute data, random 75/25 split, weather and calendar features including `dayofyear` (as in the original paper and most public projects);
- **S2:** S1 plus the load of the previous ten minutes;
- **S3:** chronological split (test = December), but a ten-minute horizon with the previous-ten-minute load;
- **S4:** this project's pipeline: hourly data, chronological split, day-ahead features.

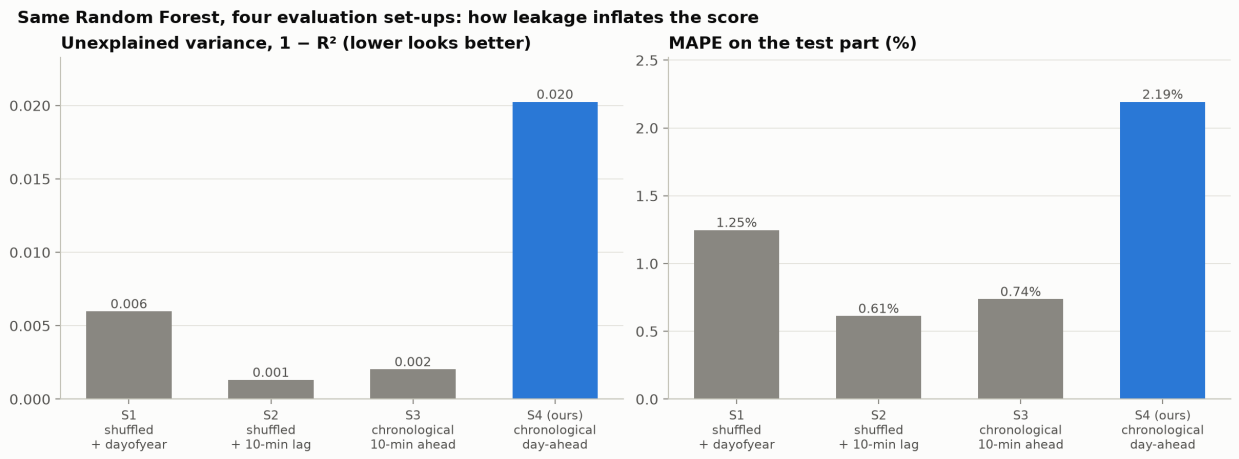


Figure 17: The same Random Forest under four evaluation set-ups: unexplained variance $1 - R^2$ (left) and MAPE (right).

S1 reaches $R^2 = 0.994$ (MAPE 1.25%), S2 0.999 (0.61%) and S3 0.998 (0.74%), while S4 reaches 0.980 (2.19%) (Figure 17; numbers in Table 21, appendix). With a random split, a test point at 20:10 has its neighbours at 20:00 and 20:20 in the training set, so the model only interpolates. In S2 the lag is built before splitting, so the load of a test row becomes the input of the training row ten minutes later: test targets leak into training. S3 is a legitimate evaluation, but of a much easier task (ten minutes ahead). The four set-ups change several things at once, so the experiment does not isolate the effect of each one. It does show that most of the high scores reported online come from the evaluation set-up rather than from a better model, and that they say nothing about next-day forecasting.

Shuffled KFold versus TimeSeriesSplit for tuning. The RF is also tuned twice on the development data, changing only the folds (20 candidates each, Table 10). Shuffled KFold estimates a CV RMSE of 2,008 kW and selects deeper trees, but the selected model reaches 2,174 kW on December, 8% worse than promised. TimeSeriesSplit gives a pessimistic estimate (3,012 kW, because of the hard July fold), yet the model it selects is better on the test set (2,044 kW). Shuffled CV thus both misjudges the error and selects a worse model.

Table 10: Tuning the same RF with shuffled KFold or TimeSeriesSplit, 20 candidates each (T12b). Optimism $= 100(\text{RMSE}_{test}/\text{RMSE}_{CV} - 1)$; errors in kW.

CV strategy	CV RMSE	December RMSE	Optimism (%)
KFold (shuffled)	2,007.6	2,173.6	8.27
TimeSeriesSplit	3,012.4	2,044.2	−32.14

10 Conclusion

This project forecasts the hourly electricity load of Tetouan one day ahead under a strictly chronological evaluation. The SVR chosen on cross-validation reaches RMSE 1,625 kW and MAPE 1.78% on the December test, halving the error of the Naive-24h baseline (skill 50.7%).

Answers to the research questions. (RQ1) For every model, the better tuned version beats both naive baselines on CV and on the test. (RQ2) Daily lags are the most useful feature group (−65% error), followed by calendar features and special days; weekly and rolling features hurt and weather adds little. (RQ3) Tuning matters most for SVR (−39%); for tree models, learning the deviation from yesterday's load matters more, because it removes their failure to extrapolate during the July heat wave. All models train in seconds. The leakage experiment shows that shuffled splits and ten-minute horizons push R^2 to 0.999 with the same model, so such scores are not comparable with day-ahead forecasting.

Limitations. The models use the measured weather of the target hour instead of weather forecasts, assume a forecast issued at 00:00 with yesterday's loads available, and are tested on a single month of a single year. One flag for all religious holidays over-corrected Mawlid by about 6%.

Future work. Use archived weather forecasts and an earlier issuance time (noon of the previous day), separate Eid from other religious holidays, test on further years and seasons, and tune the `h1` models separately.

References

[1] A. Salam and A. El Hibaoui. *Power Consumption of Tetouan City*. Year follows the repository's recommended dataset citation; donation recorded in 2023. UCI Machine Learning Repository, 2018. DOI: 10.24432/C5B034. URL: https://archive.ics.uci.edu/dataset/849/power+consumption+of+tetouan+city (visited on 10/26/2026).

[2] T. Chen and C. Guestrin. *XGBoost: A Scalable Tree Boosting System*. 2016. arXiv: 1603.02754. URL: https://arxiv.org/abs/1603.02754 (visited on 10/26/2026).

[3] A. Salam and A. El Hibaoui. "Comparison of Machine Learning Algorithms for the Power Consumption Prediction: Case Study of Tetouan City". In: *2018 6th International Renewable and Sustainable Energy Conference (IRSEC)*. Identity corroborated by the UCI dataset record; numerical details in this report follow the repository literature review. 2018. DOI: 10.1109/IRSEC.2018.8703007.

[4] N. RoSe, O. Osbourne, N. Williams, and S. S. H. Rizvi. "A Novel Optimized Variant of Machine Learning Algorithm for Accurate Energy Demand Prediction for Tetouan City, Morocco". In: *Proceedings of the International e-Conference on Advances in Computer Engineering and Communication Systems (ICACECS 2023)*. Atlantis Highlights in Computer Sciences. Atlantis Press, 2023, pp. 62–73. DOI: 10.2991/978-94-6463-314-6_7. URL: https://www.atlantis-press.com/proceedings/icacecs-23/125995751 (visited on 10/26/2026).

[5] D. G. da Silva and A. A. M. Meneses. “Comparing Long Short-Term Memory (LSTM) and Bidirectional LSTM Deep Neural Networks for Power Consumption Prediction”. In: *Energy Reports* 10 (2023), pp. 3315–3334. DOI: 10.1016/j.egyr.2023.09.175.

[6] M. T. Islam, S. A. Turja, M. T. Islam, M. M. Rahman, and A. Habib. “Forecasting Tetouan Energy Demand Employing Shift Approach in Machine-Learning: Complementing Econometric Insights”. In: *Quality & Quantity* 59 (2025). Published online in December 2024, pp. 1833–1860. DOI: 10.1007/s11135-024-02043-0.

[7] K. Alsalem. “A Hybrid Time Series Forecasting Approach Integrating Fuzzy Clustering and Machine Learning for Enhanced Power Consumption Prediction”. In: *Scientific Reports* 15, 6447 (2025). DOI: 10.1038/s41598-025-91123-8.

[8] N. Biswas, M. A. Mahdi, and M. Das Gupta. “Electricity Demand Forecasting for Tetouan, Morocco Using Darts in Machine-Learning: A Comparative Study of Forecasting Models”. In: *Results in Engineering* 30, 110289 (2026). DOI: 10.1016/j.rineng.2026.110289.

[9] scikit-learn developers. *TimeSeriesSplit: API Reference*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html (visited on 10/26/2026).

[10] J. Bergstra and Y. Bengio. “Random Search for Hyper-Parameter Optimization”. In: *Journal of Machine Learning Research* 13 (2012), pp. 281–305. URL: https://jmlr.org/papers/v13/bergstra12a.html (visited on 10/26/2026).

[11] scikit-learn developers. *Common Pitfalls and Recommended Practices*. URL: https://scikit-learn.org/stable/common_pitfalls.html (visited on 10/26/2026).

A Supplementary Material

A.1 Reproducibility

All results come from the project repository. `python run_all.py` runs the whole pipeline in order and regenerates every result table and figure; `TETOUAN_N_JOBS` sets the number of CPU threads and changes speed only. Results are reproducible because library versions are pinned (Table 11, Python 3.12), every model and search uses seed 42, and the split is defined by fixed dates. Only the timing columns change between runs.

Table 11: Recorded experiment dependencies from requirements.txt; Python 3.12 is specified by the repository. These are recorded pins, not a newly reproduced environment.

Package	Pinned version
numpy	2.5.3
pandas	3.0.6
scipy	1.18.1
scikit-learn	1.9.1
xgboost	3.4.1
statsmodels	0.15.0
shap	0.52.0
matplotlib	3.11.2
seaborn	0.13.2
jupyter	1.1.1
ipykernel	7.4.0
nbformat	5.11.1
nbclient	0.11.0

A.2 Cross-validation folds and Level 0–2 results

Table 12: Expanding-window h24 cross-validation (T02). Training starts on 8 January at 23:00 in every fold; validation begins at 00:00 and ends at 23:00.

Fold	Training ends	Train hours	Validation starts	Validation ends
0	2017-07-03	4,225	2017-07-04	2017-08-02
1	2017-08-02	4,945	2017-08-03	2017-09-01
2	2017-09-01	5,665	2017-09-02	2017-10-01
3	2017-10-01	6,385	2017-10-02	2017-10-31
4	2017-10-31	7,105	2017-11-01	2017-11-30

Table 13: Level 0 and Level 1: November validation of the seasonal baselines and the default regressors, h24 (T04). RMSE/MAE in kW; MAPE in percent.

Model	Train RMSE	Val. RMSE	Val. MAE	MAPE	R^2
XGB	812.4	2,161.4	1,598.4	2.42	0.976,0
SVR	1,567.7	2,195.2	1,601.1	2.52	0.975,3
RF	794.4	2,227.5	1,616.7	2.47	0.974,5
LR	2,670.5	2,288.7	1,734.0	2.76	0.973,1
DT	0.0	2,992.4	2,190.0	3.34	0.954,0
Naive-24h	3,497.5	3,076.4	2,080.7	3.27	0.951,4
Naive-168h	4,686.4	3,202.9	2,425.8	3.79	0.947,3

Table 14: Default-model development CV, h24 (T05). SD is the sample standard deviation over five folds, not a confidence interval. Errors in kW; MAPE in percent.

Model	RMSE mean ± SD	MAE	MAPE	R^2
LR	2,792.3 ± 623.3	2,101.0	2.95	0.965,1
RF	3,069.5 ± 1,488.8	2,168.6	2.82	0.958,2
XGB	3,264.8 ± 1,815.8	2,328.0	2.99	0.951,0
Naive-24h	3,392.4 ± 421.5	2,371.7	3.28	0.949,7
SVR	3,968.7 ± 2,891.5	2,753.4	3.44	0.922,4
DT	4,048.2 ± 1,410.4	2,915.3	3.87	0.929,4
Naive-168h	4,892.6 ± 2,297.1	3,655.4	4.92	0.888,7

Table 15: Default h24 RMSE by temporal fold (T05), in kW. Models are ordered by mean fold RMSE; fold boundaries are given in Table 12.

Model	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4
LR	3,070.8	3,307.8	3,323.4	1,970.8	2,288.7
RF	5,397.2	3,456.3	2,780.0	1,486.8	2,227.5
XGB	6,229.6	3,433.3	2,988.7	1,510.9	2,161.4
Naive-24h	4,024.4	3,562.6	3,328.4	2,970.0	3,076.4
SVR	8,774.7	4,352.6	3,040.3	1,480.6	2,195.2
DT	6,110.5	4,663.4	3,906.2	2,568.3	2,992.4
Naive-168h	6,224.7	7,205.0	6,062.4	1,768.2	3,202.9

A.3 Hyperparameter tuning

Table 16: Tuned h24 development CV (T07). RMSE and its SD are in kW. Improvement compares each row with the absolute default model of that family; Δ rows combine tuning with a target change.

Variant	Default RMSE	Tuned RMSE ± SD	Gain (%)	MAPE (%)
SVR	3,968.7	2,410.6 ± 766.3	39.26	2.42
SVR Δ	3,968.7	2,427.1 ± 773.7	38.84	2.41
RF Δ	3,069.5	2,429.3 ± 800.7	20.86	2.36
XGB Δ	3,264.8	2,441.0 ± 785.5	25.23	2.38
DT Δ	4,048.2	2,524.9 ± 851.9	37.63	2.40
LR Δ	2,792.3	2,770.0 ± 591.6	0.80	2.87
LR	2,792.3	2,792.3 ± 623.3	0.00	2.95
XGB	3,264.8	3,009.9 ± 1,470.2	7.81	2.80
RF	3,069.5	3,010.4 ± 1,588.7	1.93	2.75
DT	4,048.2	3,589.7 ± 1,756.1	11.33	3.34

Table 17: Selected h24 hyperparameters from best_params.json. Estimator-wrapper prefixes are omitted; floating values are rounded to six decimals. Both target formulations were searched independently.

Variant	Selected parameters
LR	alpha = 0.001,329
LR Δ	alpha = 98.777,003
DT	min_samples_split = 10; min_samples_leaf = 5; max_features = None; max_depth = 8
DT Δ	min_samples_split = 20; min_samples_leaf = 5; max_features = None; max_depth = 4
RF	n_estimators = 200; min_samples_leaf = 2; max_features = 0.800,000; max_depth = 30
RF Δ	n_estimators = 100; min_samples_leaf = 10; max_features = sqrt; max_depth = 30
SVR	C = 26.926,469; epsilon = 0.010,000; gamma = 0.006,339
SVR Δ	C = 138.262,322; epsilon = 0.100,000; gamma = 0.003,387
XGB	colsample_bytree = 0.798,899; learning_rate = 0.024,629; max_depth = 3; min_child_weight = 8; n_estimators = 1304; reg_lambda = 0.341,635; subsample = 0.990,646
XGB Δ	colsample_bytree = 0.831,312; learning_rate = 0.011,137; max_depth = 3; min_child_weight = 10; n_estimators = 774; reg_lambda = 9.197,754; subsample = 0.935,573

A.4 Ablation

Table 18: Feature ablation with fixed D-selected hyperparameters: mean CV RMSE in kW (T09). Missing entries were not run because the residual base is absent.

Set (columns)	RF	XGB	RF Δ	XGB Δ
A (5)	15,010.3	14,826.7	–	–
B (12)	10,040.9	9,669.3	–	–
C (15)	9,168.1	8,740.2	–	–
D (17)	3,010.4	3,009.9	2,429.3	2,441.0
E (21)	3,700.3	4,921.4	2,741.8	3,964.2
F (16)	3,759.2	5,017.1	2,887.5	3,928.4

A.5 Final test and leakage experiment

Table 19: Five largest daily MAEs for the selected h24 SVR (T08). Bias is prediction minus actual; all numerical columns are in kW.

Date	MAE	Bias	Actual mean	Predicted mean
2017-12-01	4,579.2	−3,616.7	58,574.2	54,957.4
2017-12-03	3,028.5	−2,880.5	59,192.4	56,311.9
2017-12-12	1,714.1	1,712.5	64,338.5	66,051.1
2017-12-21	1,336.2	1,118.1	64,373.9	65,492.0
2017-12-02	1,273.4	−554.7	60,104.3	59,549.7

Table 20: Refit and prediction time of the tuned h24 models (T08), single run; timings are indicative.

Model	Fit (s)	Predict 720 rows (s)	Test RMSE (kW)
LR Δ-tuned	0.003	0.001	2,217.3
DT Δ-tuned	0.014	0.001	2,005.2
RF Δ-tuned	0.096	0.025	1,761.9
SVR-tuned	3.435	0.135	1,624.6
XGB Δ-tuned	0.156	0.002	1,719.2

Table 21: Four RF diagnostic settings (T12). These rows differ in task and test distribution and must not be treated as a controlled leakage-effect estimate. Errors in kW.

Setting	Train / test rows	Features	RMSE	MAE	MAPE (%)	R^2
S1	39,311 / 13,104	13	1,325.3	854.8	1.25	0.994,0
S2	39,311 / 13,104	14	620.0	419.2	0.61	0.998,7
S3	48,095 / 4,320	13	644.0	443.3	0.74	0.998,0
S4	7,825 / 720	17	2,017.1	1,331.7	2.19	0.979,8

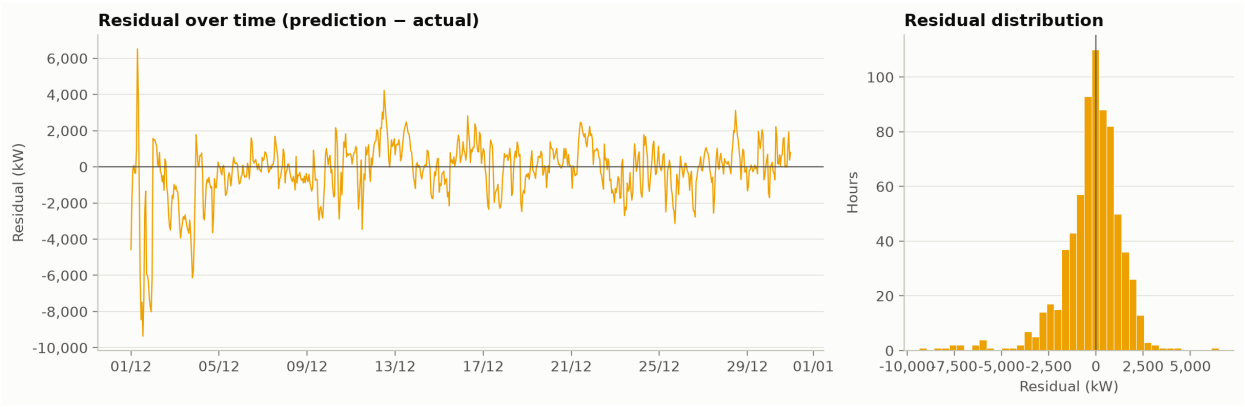


Figure 18: Residuals (prediction minus actual) of the selected SVR over December and their distribution.

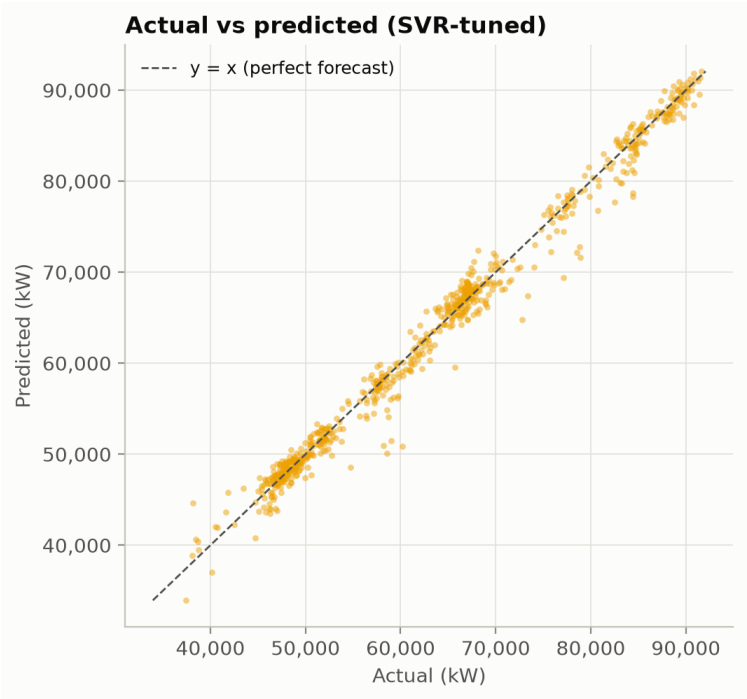


Figure 19: Actual against forecast December load of the selected SVR, with the line $y = x$.